

From PINNs to a quantity of interest

Goal-Oriented Deep Fourier Residual methods

Carlos Vera-Ciro

Goal-Oriented DFR series | Part 1

About this series

Four short decks, building the idea and then testing it honestly.

1. **The problem** (this deck): why the residual norm matters, and why global error is often the wrong target.
2. **The method**: goal-oriented DFR, a primal and an adjoint network trained together.
3. **The theory**: the error representation and the bound, with proofs.
4. **Results and limits**: what the method buys at equal compute, and where it does not help.

Takeaway: Audience: anyone comfortable with PDEs and neural networks. No prior DFR needed.

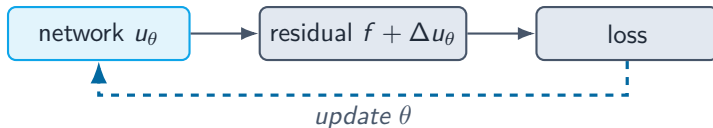
Section 1

Solving PDEs with networks

Solving a PDE with a neural network

We want the **exact solution** u^* : $-\Delta u^* = f$, with $u^* = 0$ on $\partial\Omega$. We never have it, and every symbol below is named against it.

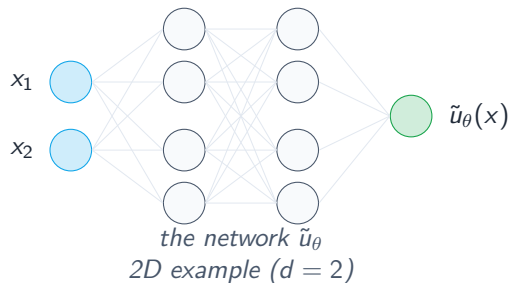
- Represent the unknown by a network u_θ with parameters θ .
- Choose θ so u_θ satisfies the equation as closely as possible, where “closely” is fixed by a **loss function**.
- The choice of loss decides how faithful the answer is.



Takeaway: Two objects to pin down: the *network* u_θ (next two slides), then the *loss* that scores it (the rest of this deck).

What the network is: a function of the coordinate

The **trainable network** is a neural field $\tilde{u}_\theta : \mathbb{R}^d \rightarrow \mathbb{R}$; every parameter θ lives in it. The **solution** we use is $u_\theta = \varphi \tilde{u}_\theta$, this network times a *fixed* cutoff φ (built on the next slide).



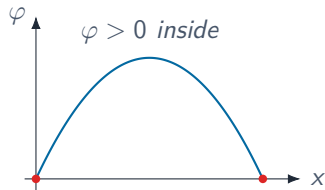
- **Inputs:** the coordinate x , d of them (1 in 1D, 2 in 2D). *Not* one input per grid point.
- **Output:** one scalar, the raw value $\tilde{u}_\theta(x)$.
- The *same* network is queried at every quadrature point (150 in 1D, 32×32 in 2D); those are not extra inputs.

Takeaway: “Degrees of freedom” later means parameter count, not a mesh: e.g. 105 for a $2 \rightarrow 8 \rightarrow 8 \rightarrow 1$ net.

Boundary conditions, built in by construction

The problem asks for $u^* = 0$ only *on* the boundary $\partial\Omega$; inside, u^* is the unknown we solve for. We enforce this exactly by construction, with no loss penalty to tune.

- $\tilde{u}_\theta$ is the trainable **network** from the last slide; on its own it is nonzero on $\partial\Omega$.
- φ is a **fixed, known** function of the *same* coordinate x that feeds the network (never trained): zero on $\partial\Omega$, positive inside. On $[0, 1]$, $\varphi(x) = x(1 - x)$; on the square, $x_1(1 - x_1)x_2(1 - x_2)$.
- The product $u_\theta = \varphi \tilde{u}_\theta$ is then zero on $\partial\Omega$ (as $\varphi = 0$) and free inside (as $\varphi > 0$).



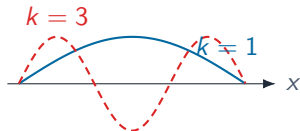
1D: zero at the ends

Takeaway: The fixed cutoff φ pins the boundary exactly and for free; the trainable network does all the real work inside.

First, read the error as a sum of sine waves

We will pick the loss to drive the residual down. Does that drive the *error* down too? To find out, write both in the natural basis for $-\Delta$ on a box: its **sine modes**.

- Any function vanishing on $\partial\Omega$ is a sum of **sine modes** $\phi_k = \sqrt{2} \sin(k\pi x)$, with $\int \phi_k^2 = 1$:
 $\varepsilon(x) = \sum_k a_k \phi_k(x)$, amplitude a_k per mode.
- They are the **eigenfunctions** of the Laplacian,
 $-\Delta \phi_k = (k\pi)^2 \phi_k$.
- So $-\Delta$ does not mix modes: it scales mode k by $(k\pi)^2$, and we can reason one mode at a time.



*higher k , higher frequency
(ϕ_k is a mode, φ the cutoff)*

Takeaway: In this basis $-\Delta$ is diagonal, scaling mode k by $(k\pi)^2$. That single fact drives both the trap and its fix.

The trap: a small residual is not a small error

Back to the loss. The obvious choice squares the pointwise residual at sample points, $\frac{1}{N} \sum_i |f(x_i) + \Delta u_\theta(x_i)|^2$, which is a **PINN**. See what it does to our modes. Write the error $\varepsilon = u_\theta - u^*$. Since $-\Delta u^* = f$, the residual *is* the error through the operator, $r = -\Delta \varepsilon$, so it scales mode k by $(k\pi)^2$:

$$\varepsilon = a \sin(k\pi x) \implies \|\varepsilon\|_{L^2} \propto |a|, \quad \|r\|_{L^2} = (k\pi)^2 |a|.$$

- **Same error, wildly different residual:** at $k = 1$ versus $k = 10$ the error is identical, yet the residual differs by $100\times$.

Takeaway: The pointwise residual over-weights high-frequency error by $(k\pi)^2$, so making it small is not the same as making the error small. We need a norm that corrects for this.

Section 2

The Deep Fourier Residual method

The DFR fix: the dual norm of the weak residual

Taylor, Pardo and Muga (2023): stop scoring the pointwise residual; measure it in the H^{-1} **dual norm** instead (weak form in Part 3).

- We saw these sine modes diagonalize $-\Delta$. The dual norm simply *reweights* each mode of the residual by $1/\lambda_k$,

$$\|R(u_\theta)\|_{V^*}^2 = \sum_k \frac{|\hat{R}_k|^2}{\lambda_k}, \quad \lambda_k = \sum_{i=1}^d (\pi k_i / L_i)^2,$$

with coefficients $\hat{R}_k$ from a **Discrete Sine Transform** (DST).

- $k = (k_1, \dots, k_d)$ is a **multi-index**, one frequency per axis; i runs over the d axes, of side lengths L_i . So in $d \geq 2$ the weight couples the axes by a *sum*, not a product.
- **The point:** on the unit interval of the last two slides ($d = 1, L = 1$), $\lambda_k = (k\pi)^2$; dividing by it undoes the amplification behind the trap, so $\|R\|_{V^*}$ tracks the energy error.

What that reweighting does to training

In modes, $\varepsilon = \sum_k a_k \phi_k$ gives $\hat{R}_k = \lambda_k a_k$, so the two losses score the *same* error very differently:

$$\mathcal{L}_{\text{PINN}} = \sum_k |\hat{R}_k|^2 = \sum_k \lambda_k^2 a_k^2, \quad \mathcal{L}_{\text{DFR}} = \sum_k \frac{|\hat{R}_k|^2}{\lambda_k} = \sum_k \lambda_k a_k^2 = \|\nabla \varepsilon\|_{L^2}^2.$$

- **The gradient is damped mode by mode:** $\nabla_{\theta} \mathcal{L}_{\text{DFR}} = 2 \sum_k \lambda_k^{-1} \hat{R}_k \nabla_{\theta} \hat{R}_k$, so high frequencies no longer dominate the update.
- **It preconditions the problem:** in the a_k the Hessian is $\text{diag}(\lambda_k)$, not $\text{diag}(\lambda_k^2)$; over K modes per axis the conditioning goes from $O(K^4)$ to $O(K^2)$.
- **The price:** the DST needs a structured grid, so training is full-batch on one fixed quadrature grid, on a box, with sines as eigenfunctions.

Takeaway: The DFR loss *is* the energy error, so the objective is better shaped, not solved: Part 4 finds plain DFR still not converged at its budget.

Error-loss equivalence (Taylor, Pardo, Muga 2023)

For a well-posed problem there are constants $\gamma, M > 0$ such that, for *any* trial function w (trained or not, so in particular our u_θ),

$$\frac{1}{M} \|R(w)\|_{V^*} \leq \|u^* - w\|_U \leq \frac{1}{\gamma} \|R(w)\|_{V^*}.$$

- **Two-sided:** the loss bounds the error above *and* below.
- Drive the DFR loss down, and the energy-norm error goes down with it.
- We prove this, and everything that follows, in Part 3.

Section 3

Quantities of interest

Global error is often not what we want

DFR gives tight, two-sided control of the *whole-field* energy error. Often that is more than we need: the object we actually care about is one **functional** of the solution, a quantity of interest (QoI):

a point value
 $u^*(x_0)$

a subdomain
average

a boundary
flux

- A small global error is *sufficient* but not *necessary* for an accurate QoI.
- Driving the global error to a tight tolerance is wasteful when only $J(u^*)$ matters.

Takeaway: We want a loss aimed at the quantity of interest, not the whole field. That is Part 2.